

Chapter 8 — The Page Hook

(hooks/useCrudPage.ts)

File: `src/features/react-crud-v4/hooks/useCrudPage.ts` **One job:** All state, all handlers, all derived data for the page. The page component calls this hook and renders — nothing else.

8.1 Why Extract Logic Into a Custom Hook?

A custom hook is just a function that starts with `use` and can call other hooks. That's it. But this simple rule unlocks a powerful pattern.

Without a custom hook:

```
// Crud.page.tsx — 200+ lines, untestable, hard to read
function CrudPage() {
  const [modal, dispatch] = useReducer(modalReducer, initialModalState);
  const [serverParams, setServerParams] = useState(undefined);
  const { data: itemsResponse, isLoading, isFetching, isError } =
useGetItems(serverParams);
  const { data: yearsResponse } = useGetYears();
  const items = useMemo(() => itemsResponse ?? [], [itemsResponse]);
  const rows = useMemo(() => items.map(mapItemDtoToRow), [items]);
  const yearOptions = useMemo(() => yearsResponse?.YearList.map(y => y.Year) ??
[], [yearsResponse]);
  const handleServerSearch = useCallback((params) => setServerParams(params), []);
  const handleCreate = useCallback(() => dispatch({ type: "OPEN_CREATE" }), []);
  const handleEdit = useCallback((id) => {
    const dto = items.find(i => i.StudentNumber === id);
    if (!dto) return;
    dispatch({ type: "OPEN_EDIT", ctx: buildEditContext(dto) });
  }, [items]);
  // ... 6 more handlers
  // ... then the JSX
  return ( ... );
}
```

With a custom hook:

```
// Crud.page.tsx — 50 lines, readable at a glance
function CrudPage() {
  const { rows, yearOptions, isLoading, modalOpen, editCtx, handleCreate,
handleEdit, ... } = useCrudPage();
  return ( ... );
}

// useCrudPage.ts — all logic, fully testable
```

```
export const useCrudPage = () => {
  // ... all the state and handlers
  return { rows, yearOptions, isLoading, modalOpen, editCtx, handleCreate,
    handleEdit, ... };
};
```

Benefits:

1. The page component is readable at a glance — you understand the feature structure in 30 seconds
2. The hook is testable with `renderHook` — no UI rendering needed
3. Multiple developers can work on the hook and the page simultaneously without conflicts
4. The hook can be reused if another page needs the same logic

8.2 The Real File — Line by Line

Imports

```
import { useCallback, useMemo, useReducer, useState } from "react";

import { useGetItems, useGetYears } from "../Services/crud.queries";
import type { GetItemsParams } from "../Services/crud.services";
import { mapItemDtoToRow } from "../helpers/mapItemDtoToRow";
import { buildEditContext } from "../helpers/buildEditContext";
import type { ItemRow } from "../helpers/mapItemDtoToRow";
import type { ItemEditContext } from "../Services/crud.types";
import type { ItemDto } from "../Services/crud.dtos";
```

All four React performance hooks are imported: `useCallback`, `useMemo`, `useReducer`, `useState`. Each has a specific use case explained below.

The Modal State Type

```
type ModalState = {
  modalOpen: boolean;
  editCtx: ItemEditContext | null;
  deleteConfirmOpen: boolean;
};
```

Why group these three fields into one state object?

These three fields are **coupled** — they change together. When you open the edit modal, you need `modalOpen: true` AND `editCtx: theItem` at the same time. When you close the modal, you need `modalOpen: false` AND `editCtx: null` at the same time.

If they were separate `useState` calls:

```
const [modalOpen, setModalOpen] = useState(false);
const [editCtx, setEditCtx] = useState(null);

// Opening the modal requires two separate state updates
setModalOpen(true);
setEditCtx(buildEditContext(dto));
```

React batches state updates in event handlers (since React 18), but in some edge cases — like updates triggered from async callbacks or third-party event systems — two separate `setState` calls can cause an intermediate render where `modalOpen` is `true` but `editCtx` is still `null`. This could cause the modal to render with no context, potentially crashing.

`useReducer` guarantees atomic updates — the state changes in a single dispatch, no intermediate renders.

The Action Union Type

```
type ModalAction =
  | { type: "OPEN_CREATE" }
  | { type: "OPEN_EDIT"; ctx: ItemEditContext }
  | { type: "CLOSE_MODAL" }
  | { type: "OPEN_DELETE_CONFIRM" }
  | { type: "CLOSE_DELETE_CONFIRM" };
```

This is a **discriminated union** — a union type where one field (the "discriminant", here `type`) distinguishes between the variants.

Why a union type? Each action has a different payload:

- `OPEN_EDIT` needs `ctx` (the item to edit)
- `OPEN_CREATE` needs nothing
- `CLOSE_MODAL` needs nothing

Without a union type, you'd have to use `any` or optional fields:

```
// BAD – no type safety
type ModalAction = {
  type: string;
  ctx?: ItemEditContext; // optional – TypeScript can't tell when it's required
};
```

With the union type, TypeScript narrows the type inside `switch` cases:

```
case "OPEN_EDIT":
  // TypeScript knows: action.ctx exists and is ItemEditContext ✓
```

```

    return { ...state, modalOpen: true, editCtx: action.ctx };

case "OPEN_CREATE":
  // TypeScript knows: action has no ctx property ✓
  // Accessing action.ctx here would be a TypeScript error

```

The Reducer

```

const modalReducer = (state: ModalState, action: ModalAction): ModalState => {
  switch (action.type) {
    case "OPEN_CREATE":
      return {
        ...state,
        modalOpen: true,
        editCtx: {
          mode: "create",
          studentId: 0,
          studentKey: "",
          activityId: 0,
          completedDate: new Date(),
          completedBy: 0,
          notes: "",
          restrictedNotes: false,
        },
      };

    case "OPEN_EDIT":
      return { ...state, modalOpen: true, editCtx: action.ctx };

    case "CLOSE_MODAL":
      return { ...state, modalOpen: false, editCtx: null };

    case "OPEN_DELETE_CONFIRM":
      return { ...state, deleteConfirmOpen: true };

    case "CLOSE_DELETE_CONFIRM":
      return { ...state, deleteConfirmOpen: false };

    default:
      return state;
  }
};

```

What is a reducer? A reducer is a pure function: $(\text{currentState}, \text{action}) \rightarrow \text{newState}$. It takes the current state and an action, and returns the new state. It never modifies the existing state — it always returns a new object.

`{ ...state, modalOpen: true }` — the spread operator creates a new object with all existing state fields, then overrides only the specified ones. This is **immutable state update** — a React requirement.

Why immutable? React uses reference equality to detect state changes. If you mutate the existing state object (`state.modalOpen = true`), the reference doesn't change, React doesn't detect the change, and the component doesn't re-render. Always return a new object.

```
// WRONG – mutates existing state
case "OPEN_CREATE":
  state.modalOpen = true; // mutating! React won't re-render
  return state;

// CORRECT – returns new object
case "OPEN_CREATE":
  return { ...state, modalOpen: true }; // new object ✓
```

OPEN_CREATE case — inline create context:

```
editCtx: {
  mode: "create",
  studentId: 0,
  studentKey: "",
  activityId: 0,
  completedDate: new Date(),
  completedBy: 0,
  notes: "",
  restrictedNotes: false,
},
```

The create context is built inline with empty defaults. `mode: "create"` tells the modal to call `createMutation` on submit. `studentId: 0` is a sentinel — the modal doesn't know which student to create for until the user selects one (or in this case, the student is already known from the row context).

default: return state — if an unknown action type is dispatched (e.g., a typo), return the current state unchanged. Prevents crashes from typos in action types.

The Hook Body

```
export const useCrudPage = () => {
  const [modal, dispatch] = useReducer(modalReducer, initialModalState);
```

useReducer(reducer, initialState) — React hook for complex state. Returns `[currentState, dispatch]`.

- `modal` — the current `ModalState`
- `dispatch` — a function to send actions to the reducer

Calling `dispatch({ type: "OPEN_CREATE" })` calls `modalReducer(modal, { type: "OPEN_CREATE" })` and re-renders the component with the new state.

```
const [serverParams, setServerParams] = useState<GetItemsParams | undefined>
(undefined);
```

Why `useState` here (not `useReducer`)? `serverParams` is independent — it doesn't change together with any other state. `useState` is simpler and correct for independent values.

`GetItemsParams | undefined` — the type is either the params object or `undefined`. `undefined` means "the user hasn't searched yet". The query hook's `enabled: !!params` uses this to prevent fetching before the user clicks Search.

```
const { data: itemsResponse, isLoading, isFetching, isError } =
useGetItems(serverParams);
const { data: yearsResponse } = useGetYears();
```

`data: itemsResponse` — destructuring with rename. `data` is React Query's generic name for the response. We rename it to `itemsResponse` to be more descriptive and avoid naming conflicts with other queries.

`isLoading` — `true` during the very first fetch (no cached data yet). Used to show a skeleton. **`isFetching`** — `true` during ANY fetch, including background refetches. Used to show a loading indicator. **`isError`** — `true` if the last fetch failed. Used to show an error message.

```
const items = useMemo<ItemDto[]>(() => itemsResponse ?? [], [itemsResponse]);
const rows = useMemo<ItemRow[]>(() => items.map(mapItemDtoToRow), [items]);
```

Two separate `useMemo` calls — why?

`items` is the raw DTO array. `rows` is the mapped row array. They are separate because:

1. `items` is needed by `handleEdit` (to find the DTO by ID for building the edit context)
2. `rows` is needed by the table (to display data)

If we combined them into one `useMemo`, `handleEdit` would need to re-map the DTO from the row, which is unnecessary work.

`itemsResponse ?? []` — if `itemsResponse` is `undefined` (query hasn't run yet or is loading), use an empty array. This prevents `items.map(...)` from crashing with "Cannot read properties of undefined".

`useMemo<ItemDto[]>(() => ..., [itemsResponse])` — the generic type `<ItemDto[]>` tells TypeScript what type the memo returns. This is optional (TypeScript can infer it) but makes the code more readable.

[itemsResponse] — the dependency array. `useMemo` only re-runs when `itemsResponse` changes reference. If `itemsResponse` is the same object as last render, the cached `items` is returned immediately.

```
const yearOptions = useMemo<string[]>(
  () => yearsResponse?.YearList.map((y) => y.Year).filter((yr) => yr !== "") ??
  [],
  [yearsResponse],
);
```

`yearsResponse?.YearList` — optional chaining. If `yearsResponse` is `undefined` (query hasn't loaded yet), stop and return `undefined`. Without `?.`, this would throw "Cannot read properties of undefined (reading 'YearList')".

`.map((y) => y.Year)` — extracts just the year string from each `YearData` object.

`.filter((yr) => yr !== "")` — removes empty strings. The API sometimes returns an empty year entry.

`?? []` — if the entire expression is `undefined` (because `yearsResponse` is undefined), use an empty array.

Handlers

```
const handleServerSearch = useCallback((params: GetItemsParams) => {
  setServerParams(params);
}, []);
```

`useCallback` — caches the function reference. Without `useCallback`, a new `handleServerSearch` function is created on every render. Since it's passed as a prop to `<ItemTable>`, `ItemTable` would re-render every time `CrudPage` re-renders — even for unrelated state changes.

[empty dependency array] — `handleServerSearch` never needs to be recreated because it only calls `setServerParams`, which is a stable reference from `useState` (React guarantees state setters are stable).

```
const handleCreate = useCallback(() => {
  dispatch({ type: "OPEN_CREATE" });
}, []);
```

`dispatch` is also a stable reference (React guarantees `useReducer`'s `dispatch` is stable). Empty dependency array is correct.

```
const handleEdit = useCallback(
  (itemId: number) => {
```

```

    const dto = items.find((i) => i.StudentNumber === itemId);
    if (!dto) return;
    dispatch({ type: "OPEN_EDIT", ctx: buildEditContext(dto) });
  },
  [items],
);

```

`[items]` — `handleEdit` depends on `items` (it calls `items.find(...)`). When `items` changes (new data loaded), `handleEdit` must be recreated to close over the new `items`. If the dependency array were `[]`, `handleEdit` would always use the `items` from the first render — a stale closure bug.

`items.find((i) => i.StudentNumber === itemId)` — finds the DTO by student number. We need the full DTO (not just the row) to build the edit context, because `ItemRow` doesn't have all the fields needed (e.g., `StudentKey`).

`if (!dto) return` — defensive guard. If the item is not found (shouldn't happen in normal usage, but defensive programming prevents crashes), do nothing instead of crashing.

The Return Object

```

return {
  // Data
  rows,
  yearOptions,
  isLoading,
  isFetching,
  isError,
  serverParams,
  hasSearched: !!serverParams,

  // Modal state
  modalOpen: modal.modalOpen,
  editCtx: modal.editCtx,
  deleteConfirmOpen: modal.deleteConfirmOpen,

  // Handlers
  handleServerSearch,
  handleCreate,
  handleEdit,
  handleCloseModal,
  handleOpenDeleteConfirm,
  handleCloseDeleteConfirm,
};

```

Why return an object (not an array)? Named properties are self-documenting and order-independent. The page component destructures only what it needs:

```

const { rows, isLoading, modalOpen, handleCreate, handleEdit } = useCrudPage();

```


If you returned an array like `useState` does, callers would have to remember the exact position of each value — fragile and unreadable with 15+ values.

hasSearched: !!serverParams — derived boolean. `!!serverParams` converts `serverParams` to boolean: `undefined` → `false`, any object → `true`. The page uses this to decide whether to show `<StatusPlaceholder>` ("Search to see results") or the actual table. Derived values belong in the hook, not in the page component.

modal.modalOpen vs just modalOpen — we destructure from `modal` to give the page flat, named properties. The page doesn't need to know that modal state comes from a reducer — it just needs `modalOpen`, `editCtx`, and `deleteConfirmOpen`.

8.3 `useState` vs `useReducer` — When to Use Which

Situation	Use
Single independent value	<code>useState</code>
Multiple values that change independently	Multiple <code>useState</code> calls
Multiple values that change together	<code>useReducer</code>
Complex update logic (many cases)	<code>useReducer</code>
Simple toggle	<code>useState</code>

In this hook:

- `serverParams` — independent, simple. `useState` ✓
- `modal` (`modalOpen` + `editCtx` + `deleteConfirmOpen`) — coupled, complex. `useReducer` ✓

8.4 `useMemo` vs `useCallback` — When to Use Which

Hook	What it caches	When to use
<code>useMemo</code>	A value (result of a computation)	Expensive calculations, stable object/array references
<code>useCallback</code>	A function	Functions passed as props to child components

```
// useMemo – caches the RESULT of calling mapItemDtoToRow on each item
const rows = useMemo(() => items.map(mapItemDtoToRow), [items]);

// useCallback – caches the FUNCTION itself
const handleEdit = useCallback((id) => { ... }, [items]);
```

The key insight: Both hooks exist to prevent unnecessary re-renders. Without them:

- `rows` would be a new array on every render → TanStack Table re-initializes → sort/filters reset
- `handleEdit` would be a new function on every render → `ItemTable` re-renders unnecessarily

8.5 The Dependency Array Rules

The dependency array is the second argument to `useMemo` and `useCallback`. It tells React when to recompute the value or recreate the function.

Rule: Include every value from the outer scope that is used inside the callback.

```
// CORRECT – items is used inside, so it's in the deps
const handleEdit = useCallback((id) => {
  const dto = items.find(i => i.StudentNumber === id); // uses items
}, [items]); // items in deps ✓

// WRONG – items is used but not in deps (stale closure bug)
const handleEdit = useCallback((id) => {
  const dto = items.find(i => i.StudentNumber === id); // uses items
}, []); // items NOT in deps X – always uses items from first render
```

Stale closure bug: If you use a value inside `useCallback` but don't include it in the dependency array, the function "closes over" the value from when it was first created. When the value changes (e.g., new data loads), the function still uses the old value. This is a subtle bug that causes incorrect behavior.

ESLint rule `react-hooks/exhaustive-deps` — automatically warns you when you're missing dependencies. Always fix these warnings — they prevent stale closure bugs.

8.6 Common Mistakes

✗ Putting all state in `useState` instead of `useReducer`

```
// WRONG – separate state can cause intermediate renders
const [modalOpen, setModalOpen] = useState(false);
const [editCtx, setEditCtx] = useState(null);

const handleEdit = (id) => {
  setModalOpen(true); // render 1: modalOpen=true, editCtx=null (CRASH if modal renders)
  setEditCtx(ctx);    // render 2: modalOpen=true, editCtx=ctx (correct)
};

// CORRECT – atomic update, no intermediate renders
dispatch({ type: "OPEN_EDIT", ctx }); // single render with both changes
```

✗ Missing dependencies in `useCallback`

```
// WRONG – stale closure: always uses items from first render
const handleEdit = useCallback((id) => {
  const dto = items.find(i => i.StudentNumber === id);
}, []); // missing items!

// CORRECT
const handleEdit = useCallback((id) => {
  const dto = items.find(i => i.StudentNumber === id);
}, [items]); // items included ✓
```

✗ Not using `useMemo` for the rows array

```
// WRONG – new array every render → TanStack Table re-initializes
const rows = items.map(mapItemDtoToRow);

// CORRECT – cached, only recomputes when items changes
const rows = useMemo(() => items.map(mapItemDtoToRow), [items]);
```

✗ Putting business logic in the page component

```
// WRONG – logic in the page, untestable
function CrudPage() {
  const [modalOpen, setModalOpen] = useState(false);
  const handleEdit = (id) => { ... };
  return <ItemTable onEdit={handleEdit} />;
}

// CORRECT – logic in the hook, testable
function CrudPage() {
  const { modalOpen, handleEdit } = useCrudPage();
  return <ItemTable onEdit={handleEdit} />;
}
```

Next: [Chapter 9 — Components](#)